LLD OF SPLITWISE

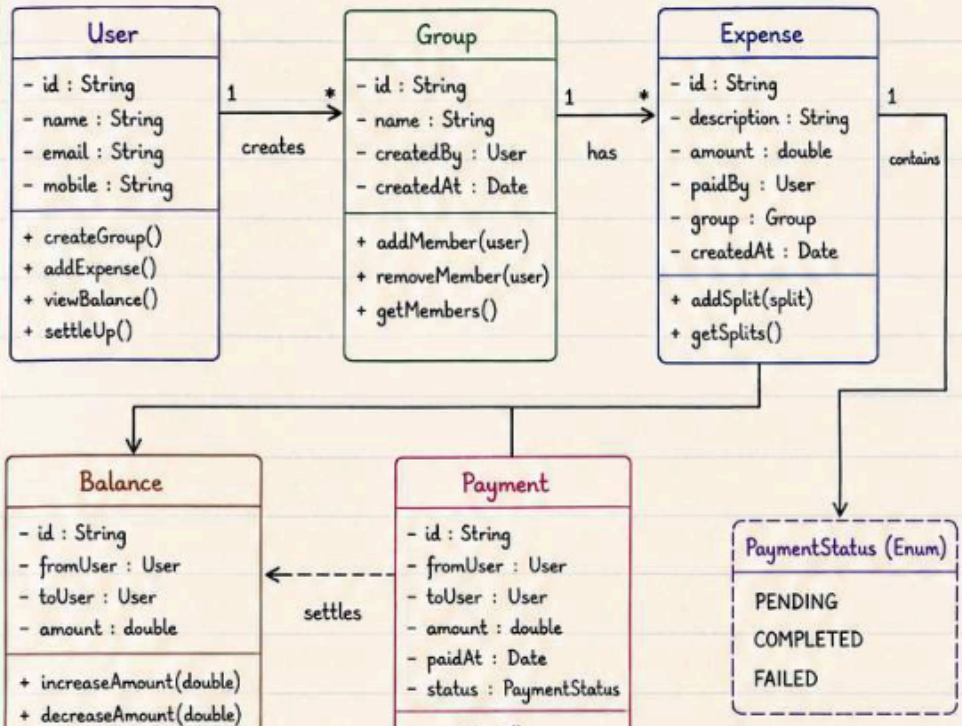
WHAT IS SPLITWISE ?

An app to help friends split expenses, track balances and settle up.

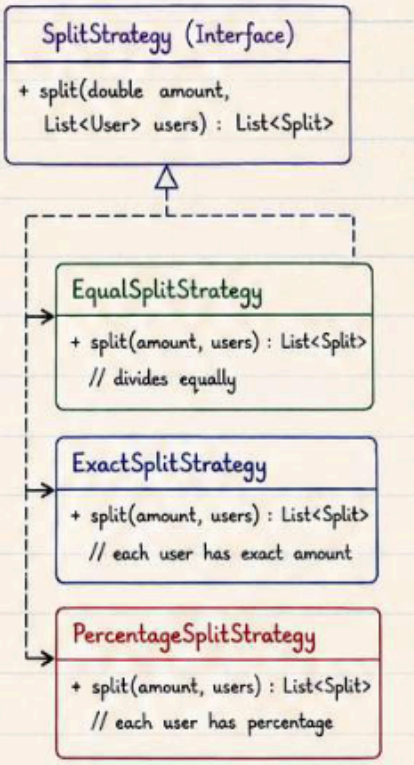
CORE FEATURES

- Create Group
- Add Members
- Add Expense
- Split Expense (Equal/Exact/Percentage)
- View Balances
- Settle Up

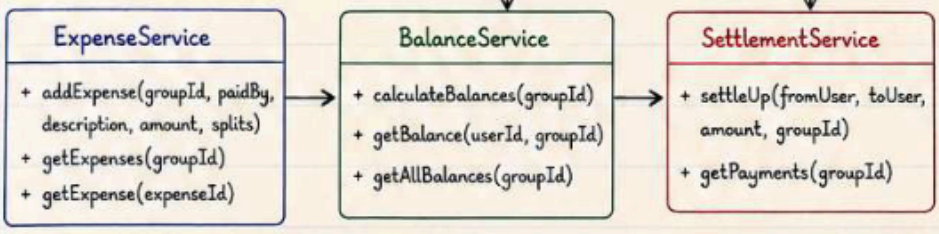
1) ENTITIES / CLASSES



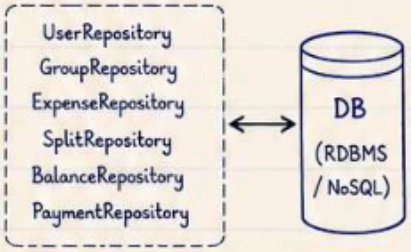
2) SPLIT STRATEGY (Strategy Pattern)



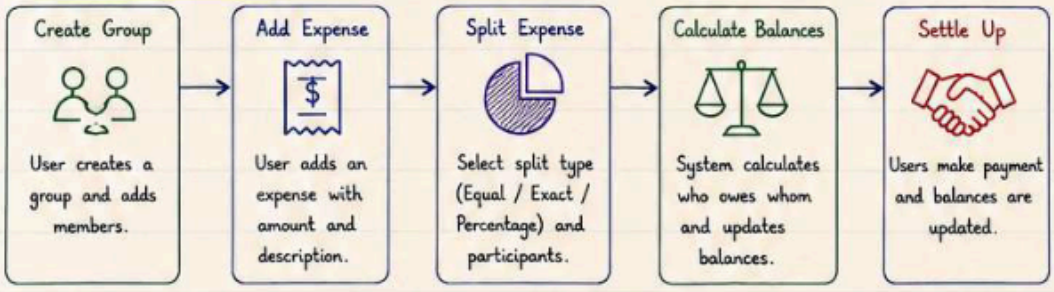
3) SERVICE LAYER



4) REPOSITORY LAYER



5) HIGH LEVEL FLOW



6) RELATIONSHIPS

1	One
*	Many
1 —— *	One to Many
* —— *	Many to Many
---->	Dependency
——▷	Inheritance

KEY NOTES

- ✓ Balances are always between two users in a group.
- ✓ Expenses never store balances, they only create splits.
- ✓ SplitStrategy makes system extensible for new split types.
- ✓ All services are stateless and interact via repositories.

DESIGN PATTERNS USED

- Strategy Pattern — For different split types
- Repository Pattern — For data access layer
- Single Responsibility — Each service has single responsibility
- Separation of Concerns — Controller, Service, Repository separation

LLD of SPLITWISE

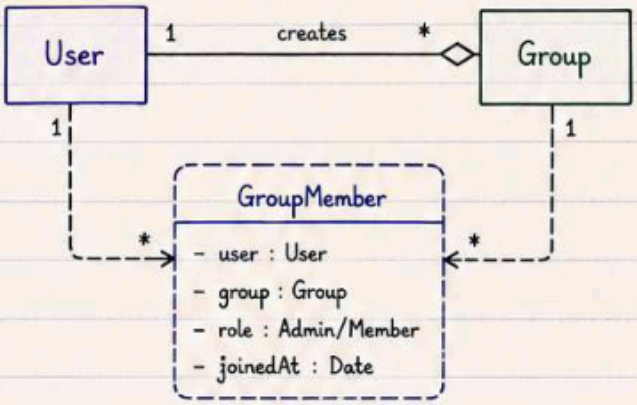
2) USER & GROUP

Users create groups and add friends to the group.
All expenses and balances are within a group.

A) CONCEPT

- User is the core actor.
- A User can be part of multiple groups.
- A Group has multiple members.
- One user creates a group.

Relationship Diagram



Notes

Many-to-Many relationship between User and Group is resolved using GroupMember.

B) CLASS DESIGN

User

- id : String
- name : String
- email : String
- mobile : String
- createdAt : DateTime

+ createGroup(name : String) : Group
+ getGroups() : List<Group>
+ getTotalBalance(groupId : String) : double

Group

- id : String
- name : String
- createdBy : User
- createdAt : DateTime

+ addMember(user : User, role : Role)
+ removeMember(user : User)
+ getMembers() : List<User>
+ getMemberCount() : int

GroupMember

- id : String
- user : User
- group : Group
- role : Role (ADMIN / MEMBER)
- joinedAt : DateTime

+ isAdmin() : boolean
+ isMember() : boolean

enum Role

ADMIN
MEMBER

Key Points

✓ Admin can add/remove members and expenses.
✓ Members can add/view expenses and balances.
✓ All balances are calculated within a group.

Example

User A creates group "Trip to Manali"
Adds B, C, D as members.
All expenses in this trip group will be tracked here.

C) OPERATIONS FLOW

1) Create Group

User (A) createGroup("Trip")
↓
Group created by A
↓
A is added as ADMIN

2) Add Member

Admin calls addMember(B, MEMBER)
↓
GroupMember created (B, Group, MEMBER)
↓
B can now add/view expenses

D) KEY METHODS SUMMARY

- User.createGroup(name) → Creates a new group
- Group.addMember(user, role) → Adds user to group
- Group.removeMember(user) → Removes user from group
- Group.getMembers() → Returns all members
- User.getTotalBalance(groupId) → Returns net balance of user in a group

LLD of SPLITWISE

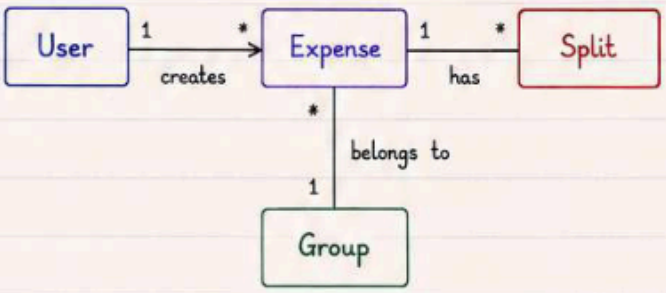
3) EXPENSE & SPLIT

Expenses are created inside a group and split among the participants using different split strategies.

A) CONCEPT

- One user pays the entire amount.
- Expense is split among participants.
- Each participant's share is recorded as a Split.
- Splits are used to calculate balances.

B) RELATIONSHIP DIAGRAM



Notes

Expense belongs to a Group.
Each Expense has multiple Splits.

C) CLASS DESIGN

Expense

- id : String
- description : String
- amount : double
- paidBy : User
- group : Group
- createdAt : DateTime

+ addSplit(split : Split)
+ getSplits() : List<Split>
+ getTotalAmount() : double

Split

- id : String
- amount : double
- user : User
- expense : Expense
- splitType : SplitType (EQUAL / EXACT / PERCENTAGE)

+ getUser() : User
+ getAmount() : double
+ getSplitType() : SplitType

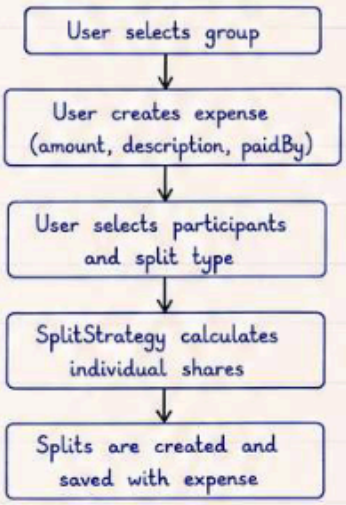
SplitType (Enum)

- EQUAL
- EXACT
- PERCENTAGE

Example

A pays ₹1200 for B, C, D
Split type : EQUAL
Each gets ₹400

D) EXPENSE FLOW



E) SPLIT EXAMPLES

Split Type	Scenario	Result (Shares)
EQUAL	Amount = ₹900 Users = A, B, C	A = ₹300 B = ₹300 C = ₹300
EXACT	Amount = ₹1000 A pays for B, C, D	B = ₹400 C = ₹300 D = ₹300
PERCENTAGE	Amount = ₹2000 A pays Percentages: B-50%, C-30%, D-20%	B = ₹1000 (50%) C = ₹600 (30%) D = ₹400 (20%)

F) KEY POINTS

- ✓ Expense is always created within a group.
- ✓ One user pays the full amount.
- ✓ Splits determine how the amount is divided among participants.
- ✓ All splits of an expense must sum up to the total amount.

LLD of SPLITWISE

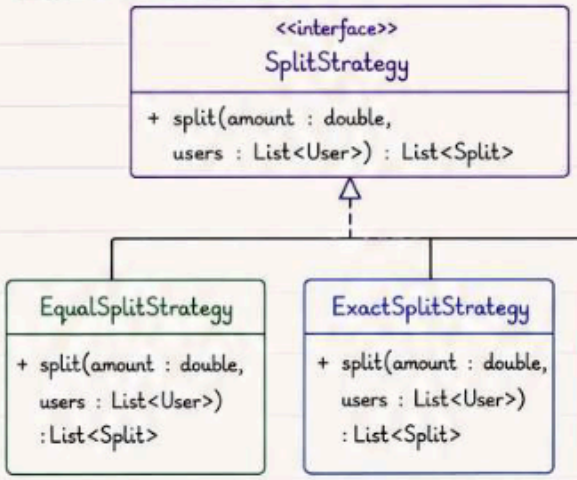
4) SPLIT STRATEGY (STRATEGY PATTERN)

SplitStrategy decides how the expense amount is divided among participants.

A) CONCEPT

- Different split types have different logic.
- Strategy Pattern allows us to add new split types without changing existing code.
- Context (ExpenseService) uses appropriate strategy at runtime.

B) CLASS DIAGRAM



Notes

New split types can be added by creating new class implementing SplitStrategy.

C) INTERFACE

```
public interface SplitStrategy {
    List<Split> split(double amount, List<User> users);
}
```

D) IMPLEMENTATIONS

1) Equal Split

```
public class EqualSplitStrategy implements SplitStrategy {
    public List<Split> split(double amount, List<User> users) {
        double share = amount / users.size();
        List<Split> splits = new ArrayList<>();
        for (User user : users) {
            splits.add(new Split(user, share)); // each user gets equal share
        }
        return splits;
    }
}
```

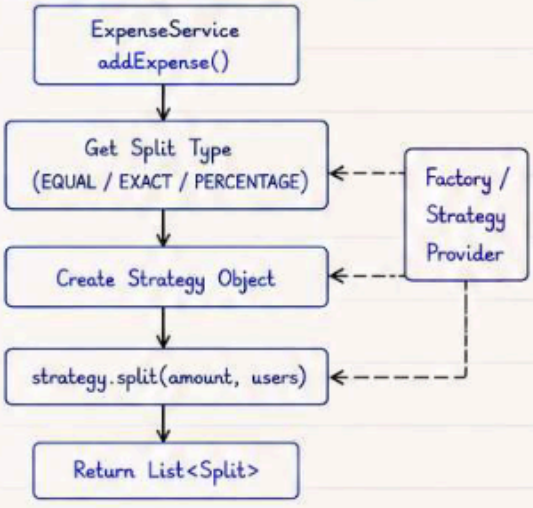
2) Exact Split

```
public class ExactSplitStrategy implements SplitStrategy {
    public List<Split> split(double amount, List<User> users) {
        // amount is already provided for each user
        // assume users have exact amounts in map (user -> amount)
        List<Split> splits = new ArrayList<>();
        for (User user : users) {
            double userAmount = getExactAmount(user); // from request
            splits.add(new Split(user, userAmount));
        }
        return splits;
    }
}
```

3) Percentage Split

```
public class PercentageSplitStrategy implements SplitStrategy {
    public List<Split> split(double amount, List<User> users) {
        List<Split> splits = new ArrayList<>();
        for (User user : users) {
            double percent = getUserPercentage(user); // from request
            double userAmount = amount * percent / 100.0;
            splits.add(new Split(user, userAmount));
        }
        return splits;
    }
}
```

E) EXPENSE SERVICE USAGE



KEY POINTS

- ✓ SplitStrategy is the heart of Splitwise.
- ✓ Different split types have different calculation logic.
- ✓ Adding a new split type does not affect existing code.
- ✓ ExpenseService depends on abstraction, not on concrete classes.

LLD of SPLITWISE

5) BALANCE & SETTLEMENT

Balances show who owes whom and how much.
Settlement is the payment made to clear the balance.

A) CONCEPT

- After every expense, balances are updated.
- A positive balance means "to receive".
- A negative balance means "to pay".
- Settlement reduces the balance to zero.

B) BALANCE MODEL

Balance

- id : String
- fromUser : User (who owes)
- toUser : User (who should receive)
- amount : double

+ increaseAmount(double)
+ decreaseAmount(double)
+ getAmount() : double

Balance Representation

From (Owes)	To (Should Receive)	Amount
B	A	₹ 400
C	A	₹ 400
D	A	₹ 200

Positive for A, Negative for others.

C) BALANCE CALCULATION EXAMPLE

Expense
A paid ₹1200 for A, B, C, D
Split Type : EQUAL

→ Each gets ₹1200 / 4 = ₹300

Individual Share

User	Share (₹)
A	300
B	300
C	300
D	300

→ A paid ₹1200

Net Position

A : +900 (to receive)
B : -300 (to pay)
C : -300 (to pay)
D : -300 (to pay)

→ Balances Generated

Balances (Who owes whom?)

B owes A ₹300
C owes A ₹300
D owes A ₹300

D) SETTLEMENT FLOW

1. B owes A ₹300 — B ⇢ (₹300, Pending) ⇢ A
2. B pays A ₹300 — B → (₹300) → A
3. Balance Updated — B → (₹0, Settled) → A
4. Remaining Balances
   - C owes A ₹300
   - D owes A ₹300

E) SETTLEMENT SERVICE

Class Diagram

SettlementService

+ settle(fromUser : User, toUser : User, amount : double, groupId : String)
+ getPayments(groupId : String) : List<Payment>

Payment

- id : String
- fromUser : User
- toUser : User
- amount : double
- paidAt : Date
- groupId : String

+ markPaid()
+ getStatus() : PaymentStatus

PaymentStatus (Enum)

PENDING
COMPLETED
FAILED

Sample Settlement Scenario

Balances before settlement:
B owes A ₹300
C owes A ₹300
D owes A ₹300

↓

After B pays A ₹300:
B → A : ₹0 (Settled)
C → A : ₹300 (Pending)
D → A : ₹300 (Pending)

KEY POINTS

- ✓ Balances are always maintained between two users.
- ✓ Positive balance means "to receive", negative means "to pay".
- ✓ Settlement creates a Payment record and updates balances.
- ✓ After full settlement, all balances for the group should be zero.

METHODS SUMMARY

- BalanceService.calculateBalances(groupId) → Calculates and updates balances
- BalanceService.getBalance(userId, groupId) → Get net balance of user
- BalanceService.getAllBalances(groupId) → Get all balances (who owes whom)
- SettlementService.settle(from, to, amount, groupId) → Settle balance
- SettlementService.getPayments(groupId) → Get settlement history

LLD of SPLITWISE

6) SERVICES & COMPLETE FLOW

A) SERVICE LAYER

Services contain the business logic and orchestrate the flow between different components.

ExpenseService
- + addExpense(groupId, paidBy, amount, description, splits)
- + getExpenses(groupId)
- + getExpense(expenseId)

BalanceService
- + calculateBalances(groupId)
- + getBalance(userId, groupId)
- + getAllBalances(groupId)

SettlementService
- + settle(fromUser, toUser, amount, groupId)
- + getPayments(groupId)
- + getSettlementHistory(groupId)

GroupService
- + createGroup(name, userId)
- + addMember(groupId, user, role)
- + removeMember(groupId, user)
- + getGroupDetails(groupId)

B) COMPLETE REQUEST FLOW (Add Expense)

User adds Expense → ExpenseService addExpense() → Select SplitStrategy (Equal/Exact/Percentage) → Strategy calculates individual shares → Expense & Splits are saved → BalanceService updates balances → Balances are available to users

Flow Steps
1. Validate group and participants
2. Choose appropriate SplitStrategy
3. Calculate individual shares
4. Save Expense and Splits
5. Recalculate and update balances
6. Users can view updated balances

C) REPOSITORY LAYER

- UserRepository
- GroupRepository
- ExpenseRepository
- SplitRepository
- BalanceRepository
- PaymentRepository

↔ DB (RDBMS / NoSQL)

Responsibilities
- Handle all data persistence.
- Provide CRUD operations.
- Isolate data access from business logic.
- Easy to switch DB or add caching.

D) END TO END FLOW (Example)

①	②	③	④	⑤	⑥	⑦
A adds expense ₹1200 for A, B, C, D (Equal Split)	Each gets share = ₹300	Balances calculated	B pays A ₹300	Balances updated	C pays A ₹300	Final balances
A	A ₹300, B ₹300, C ₹300, D ₹300	B owes A ₹300 C owes A ₹300 D owes A ₹300	B → A ₹300	C owes A ₹300 D owes A ₹300	C → A ₹300	D owes A ₹300

After D pays A ₹300 → All balances become zero. Settlement complete!

E) DESIGN PATTERNS USED

Strategy Pattern (for Split Types)

SplitStrategy → Equal Split, Exact Split, Percent Split

Factory Pattern (for Strategy Creation)

SplitStrategyFactory: getStrategy(type) → Equal Split, Exact Split, Percent Split

Repository Pattern (for Data Access)

Service Layer → Repository Layer (Repositories) → Database

Single Responsibility (each service has one job)
- ExpenseService → Expenses
- BalanceService → Balances
- SettlementService → Payments
- GroupService → Groups

Separation of Concerns
- Controller → API Handling
- Service → Business Logic
- Repository → Data Access
- DB → Data Storage

KEY TAKEAWAY

- ✓ Strategy Pattern makes it easy to add new split types.
- ✓ Balances are maintained between any two users.
- ✓ Settlement clears balances and keeps the system consistent.
- ✓ Clean separation of layers makes the system maintainable and scalable.

LLD INTERVIEW QUESTIONS

1. How will you add a new split type?
2. How do you ensure that total split amount equals expense amount?
3. How will you handle concurrent expenses in the same group?
4. Where should balance calculation happen and why?
5. How will you handle a user leaving a group?
6. How will you design the database schema for this system?